Deep Neural Networks for Survival Analysis with survdnn

Architecture, loss functions, and prediction in a native R torch implementation.

Imad EL BADISY

2026-08-24

Table of contents

Right-censored survival data $\{(t_i, \delta_i, x_i)\}_{i=1}^n$ pair an observed time $t_i = \min(T_i, C_i)$ and an event indicator $\delta_i = \mathbb{1}(T_i \leq C_i)$ with a covariate vector $x_i \in \mathbb{R}^p$. The Cox proportional hazards model expresses the hazard as $\lambda(t \mid x) = \lambda_0(t) \exp(\beta^\top x)$, restricting covariate effects to a linear, time-constant log-hazard ratio. `survdnn` replaces the linear predictor $\beta^\top x$ with a multilayer perceptron $f_\theta(x)$, keeping the estimation machinery around it (partial likelihood, Breslow baseline, time-dependent risk) intact, and extends the same replacement to non-proportional-hazards and accelerated failure time formulations.[1] The package is implemented natively in R on top of `torch`, avoiding a Python backend.[2]

[1] `survdnn` package documentation, CRAN: https://CRAN.R-project.org/package=survdnn.
[2] EL BADISY, I. (2026). *SurvDNN: Survival Deep Learning Models for Tabular Data*. The R Journal.

Architecture

For an input of dimension p and hidden layer sizes $h_1, \dots, h_L$, the network is a sequential composition of blocks

$$z^{(0)} = x, \qquad z^{(l)} = \text{Dropout}\Big(\phi\big(\text{BN}(W^{(l)} z^{(l-1)} + b^{(l)})\big)\Big), \qquad l = 1, \dots, L,$$

where ϕ is one of `relu`, `leaky_relu`, `tanh`, `sigmoid`, `gelu`, `elu`, or `softplus`, BN is an optional batch normalization layer, and dropout is applied after each block. The output layer is a single linear map $g_\theta(z^{(L)}) = W^{(L+1)} z^{(L)} + b^{(L+1)} \in \mathbb{R}$, so the whole network is $f_\theta(x) = g_\theta(z^{(L)}(x))$. Covariates are standardized before entering the network from training-set moments, and those moments are stored on the fitted object and reapplied, not recomputed, at prediction time.

The scalar output $f_\theta(x)$ has no fixed meaning on its own: it is a log-risk score under the two Cox losses, a location parameter on the log-time scale under AFT, and a time-dependent log-risk score under Cox-Time. Everything downstream, including how a fitted network becomes a survival curve, depends on which of these it is.

Loss functions

Cox partial likelihood

Under proportional hazards, $\lambda(t \mid x) = \lambda_0(t) e^{\eta}$ with $\eta = f_\theta(x)$. Cox's partial likelihood avoids ever estimating $\lambda_0(t)$ by conditioning on which subject, among those still at risk, is the one who fails at each observed event time. For a risk set $R(t_i) = \{j : t_j \geq t_i\}$, the probability that it is subject i who fails, given that exactly one failure occurs at t_i, is

$$\Pr(i \text{ fails} \mid \text{one failure in } R(t_i)) = \frac{\lambda_0(t_i) e^{\eta_i}}{\sum_{j \in R(t_i)} \lambda_0(t_i) e^{\eta_j}} = \frac{e^{\eta_i}}{\sum_{j \in R(t_i)} e^{\eta_j}},$$

with $\lambda_0(t_i)$ canceling top and bottom, which is exactly why no baseline hazard model is needed to train on this objective. `survdnn` trains f_θ against the negative partial log-likelihood, averaged over events,

$$\ell_{\text{cox}}(\theta) = -\frac{1}{n_e} \sum_{i:\, \delta_i = 1} \left(f_\theta(x_i) - \log \sum_{j \in R(t_i)} \exp f_\theta(x_j) \right),$$

where n_e is the number of events. The inner sum is computed with `torch_logcumsumexp()` on time-sorted linear predictors, so risk-set membership falls out of the sort order and the log-sum-exp stays numerically stable rather than overflowing. An L2 variant, `"cox_l2"`, adds a penalty directly on the predicted risk scores rather than on the network weights,

$$\ell_{\text{cox-l2}}(\theta) = \ell_{\text{cox}}(\theta) + \lambda \cdot \frac{1}{n} \sum_{i=1}^{n} f_\theta(x_i)^2, \qquad \lambda = 10^{-3}.$$

Penalizing the output constrains what the network is allowed to predict regardless of how many parameters produced it, which is a more direct fix for the instability that a few extreme risk scores cause in every risk-set sum above, compared to ordinary weight decay acting on the parameters themselves.

Accelerated failure time

Under the AFT loss, the network output $\mu_\theta(x)$ is the location parameter of a log-normal model for the event time,

$$\log T = c + \mu_\theta(x) + \sigma Z, \qquad Z \sim \mathcal{N}(0, 1),$$

where c is a location offset fixed, before training, at the mean log event time among observed events in the training data (stored as `aft_loc`), and $\sigma > 0$ is a single global scale parameter optimized jointly with θ through the reparameterization $\sigma = \exp(\log \sigma)$, which keeps it positive without a constrained optimizer. Centering on c keeps the network's job restricted to a zero-centered deviation instead of also having to represent the population-average log-time through its bias term. With $z_i = (\log t_i - c - \mu_\theta(x_i))/\sigma$, the censored negative log-likelihood combines an exact density term for events with a survival term for censored observations,

$$\ell_{\text{aft}}(\theta, \sigma) = \frac{1}{n} \sum_{i=1}^{n} \Big[\delta_i \big(\log t_i + \log \sigma + \tfrac{1}{2} z_i^2 \big) - (1 - \delta_i) \log S_Z(z_i) \Big], \qquad S_Z(z) = 1 - \Phi(z),$$

the standard parametric censored-data likelihood, with the location term replaced by a network.

Cox-Time

The Cox-Time loss extends the Cox partial likelihood to a time-varying log-risk function $g_\theta(t, x)$, taking time as an explicit network input rather than treating risk as fixed per subject.[3] The contribution of an event at t_i becomes

[3] Kvamme, H., Borgan, Ø., and Scheel, I. (2019). Time-to-event prediction with neural networks and Cox regression. *Journal of Machine Learning Research*, 20(129), 1-30.

$$\ell_{\text{coxtime}}(\theta) = -\frac{1}{n_e} \sum_{i:\,\delta_i=1} \left(g_\theta(t_i, x_i) - \log \sum_{j \in R(t_i)} \exp\, g_\theta(t_i, x_j) \right),$$

which differs from the Cox loss only in that the denominator re-evaluates the network at t_i for every subject at risk, instead of reusing one risk score per subject computed once. This is what buys freedom from proportional hazards, and also what makes the loss expensive: building the denominator at every event time requires one forward pass per (event time, at-risk subject) pair, so training cost scales with the number of events rather than staying fixed as it does for plain Cox. Risk-set membership is always computed on raw observed time, while the time fed into the network is centered and scaled for optimization stability, exactly mirroring how covariates are standardized before entering the network.

Prediction

`predict.survdnn()` returns one of three things depending on `type`. `type = "lp"` returns the raw network output as interpreted by the training loss. `type = "survival"` returns predicted survival probabilities at a set of times. `type = "risk"` returns $1 - S(t)$ at a single time point. The mechanics of getting from a trained network to an actual curve differ completely across the four losses, because only AFT trains a fully specified parametric model; the others train a relative or time-varying risk score that still needs a baseline hazard attached.

For Cox and Cox-L2, survival curves are not read directly off the network. Predictions require refitting a one-covariate `coxph()` on the training linear predictors and extracting its Breslow cumulative baseline hazard $\hat{H}_0(t)$ via `survival::basehaz()`, interpolated at the requested times. This reuses `coxph()`'s existing risk-set summation and tie-handling rather than reimplementing the Breslow estimator directly against the network's gradients:

$$\hat{S}(t \mid x) = \exp\big(- \hat{H}_0(t) \exp(f_\theta(x)) \big).$$

For AFT, survival probabilities follow directly from the fitted log-normal model, with no post-hoc refitting step, using the stored c and the learned $\hat{\sigma}$ from training:

$$\hat{S}(t \mid x) = 1 - \Phi\left(\frac{\log t - c - \mu_\theta(x)}{\hat{\sigma}} \right).$$

For Cox-Time, there is no closed-form Breslow shortcut, since the baseline hazard is only meaningful jointly with a time-varying g_θ. Instead, baseline hazard increments are estimated at each observed training event time t_k from the training risk sets,

$$d\hat{H}_0(t_k) = \frac{dN(t_k)}{\sum_{j \in R(t_k)} \exp\, g_\theta(t_k, x_j)},$$

a discrete Nelson-Aalen-style ratio of events at t_k to risk-weighted exposure at t_k, and cumulative hazard at a query time is obtained by summing increments at all event times up to that point, each evaluated through the new subject's own network output at that grid point:

$$\hat{H}(t \mid x) = \sum_{t_k \le t} \exp\, g_\theta(t_k, x)\, d\hat{H}_0(t_k), \qquad \hat{S}(t \mid x) = \exp\big(-\hat{H}(t \mid x)\big).$$

This requires one forward pass per event time per subject on both the training side, to build $d\hat{H}_0$, and the prediction side, which is the direct computational cost of letting the risk score vary with time.

The Cox and Cox-L2 losses share exactly this same prediction path: nothing downstream of the frozen network can tell them apart, since the output-level penalty in `"cox_l2"` only changes what happens during training, not the Breslow step above. AFT and Cox-Time, in contrast, each need their own prediction formula because their trained scalar means something structurally different.

Training

Fitting proceeds by standard gradient-based optimization: for each epoch, a forward pass computes the loss on the full training set, `backward()` accumulates gradients, and one of `adam`, `adamw`, `sgd`, `rmsprop`, or `adagrad` updates the parameters, with `adam` and `adamw` defaulting to a small weight decay of `1e-4` unless overridden. Training is full-batch rather than mini-batch, so one epoch is one optimizer step over the entire dataset. Callbacks receive the epoch index and current loss after each step and can halt training early, as with `callback_early_stopping()`. Missing values in the model variables are either dropped, `na_action = "omit"`, or treated as an error, `na_action = "fail"`, before any tensor is built. Reproducibility across R and torch's separate random number generators is handled by a single `.seed` argument, since torch keeps its own generator state for weight initialization and dropout that `set.seed()` alone does not touch.

Evaluation

Three metrics are implemented directly against predicted survival matrices rather than delegated to an external package: the concordance index, `cindex_survmat()`, evaluated at a fixed horizon by comparing $1 - S(t^\star \mid x)$ across all admissible pairs; the IPCW-weighted Brier score, `brier()`, at a single time point, using a Kaplan-Meier estimate of the censoring distribution as weights; and the integrated Brier score, `ibs_survmat()`, a trapezoidal integral of the Brier score over a grid of times, normalized by the width of the grid. `evaluate_survdnn()` wraps these for a fitted model, and `cv_survdnn()` repeats fit-and-evaluate over stratified folds, stratifying on the event indicator so that fold sizes stay balanced across censoring rates rather than across time-to-event values.

Example

A model is fit with the same formula interface as `survival::coxph()`.

```
library(survdnn)
library(survival)

veteran <- survival::veteran

mod <- survdnn(
  Surv(time, status) ~ age + karno + celltype,
  data       = veteran,
  hidden     = c(16, 8),
  loss       = "cox",
  epochs     = 100,
  lr         = 1e-3,
  .seed      = 1,
  verbose    = FALSE
)

mod
```

Survival curves at a grid of times follow the same `predict()` interface used for `coxph` and `survfit` objects.

```
pred <- predict(mod, newdata = veteran, type = "survival", times = c(30, 90, 180))
round(head(pred), 3)
```

```
   t=30  t=90 t=180
1 0.806 0.573 0.259
2 0.847 0.652 0.354
3 0.835 0.629 0.325
4 0.832 0.622 0.316
5 0.844 0.645 0.346
6 0.684 0.376 0.093
```

Model assessment

Discrimination and calibration are read off the same predicted matrix.

```r
y <- model.response(model.frame(mod$formula, veteran))

cindex_survmat(y, pred, t_star = 180)
```

```
 c-index
0.733631
```

```r
ibs_survmat(y, pred, times = c(30, 90, 180))
```

```
     ibs
0.177018
```

Cross-validated performance follows the same call structure, now with the model refit per fold.

```r
res <- cv_survdnn(
  Surv(time, status) ~ age + karno + celltype,
  data    = veteran,
  times   = c(30, 90, 180),
  metrics = c("cindex", "ibs"),
  folds   = 3,
  .seed   = 42,
  hidden  = c(16, 8),
  epochs  = 50,
  verbose = FALSE
)

summarize_cv_survdnn(res)
```

```
# A tibble: 2 x 5
  metric  mean      sd lower upper
  <chr>  <dbl>   <dbl> <dbl> <dbl>
1 cindex 0.601 0.0515  0.542 0.659
2 ibs    0.209 0.00615 0.202 0.216
```

Practical notes

The AFT and Cox-Time losses cost more per epoch than plain Cox: AFT adds one learnable global parameter, and Cox-Time evaluates the network on a full $n_e \times n$ grid of event-time and

subject pairs at every step, so training cost grows with the number of events rather than staying constant. Batch normalization interacts poorly with very small batches; since `survdnn` trains full-batch by default this is only a concern for small datasets, where turning batch normalization off, `batch_norm = FALSE`, is often more stable than leaving it on. Dropout and weight decay are the two regularizers exposed directly, and `tune_survdnn()` and `gridsearch_survdnn()` search over these together with architecture and learning-rate settings under cross-validation.

The proportional-hazards structure of the Cox and Cox-L2 losses has a consequence worth flagging for any downstream curve-shape analysis: under either loss, $\hat{S}_i(t) = \hat{S}_0(t)^{\exp(\eta_i)}$ is a scalar power transform of one shared baseline curve, so any two predicted curves from the same fitted model differ only in scale, never in shape. Methods that cluster or compare predicted curves by their shape, such as `unsurv`, are only doing genuinely curve-native work when paired with `survdnn` fit under `"aft"` or `"coxtime"`, where curve shape is free to vary across individuals rather than being tied to a single common baseline.